1. Explain Outer Join with example. (Long Answer)

An **Outer Join** is a type of join that returns all the matching rows between two tables, **plus** the non-matching rows from one or both tables. Unlike inner join, which shows only common values, outer join helps to display **complete information**, even if related data is missing.

Types of Outer Join

1. **Left Outer Join**
   - Returns all records from the left table and matching records from the right table.
   - If no match, NULL is shown.
2. **Right Outer Join**
   - Returns all records from the right table and matching from left.
   - No match shows NULL.
3. **Full Outer Join**
   - Returns all matching and non-matching rows from **both** tables.

Example

Consider two tables:

Student

SID	SName
1	Anil
2	Sunil
3	Neha

Marks

SID	Marks
1	90
3	80

Query

```
SELECT s.sid, s.sname, m.marks
```

```
FROM Student s
LEFT OUTER JOIN Marks m
ON s.sid = m.sid;
```

Result

SID	Name	Marks
1	Anil	90
2	Sunil	NULL
3	Neha	80

This shows even students who don't have marks.

---

2. Explain Domain Integrity Constraint with syntax & example. (Long)

Domain integrity ensures that values stored in a column **belong to a specific domain**, meaning they must follow the rules like datatype, range, format, etc.

How domain integrity is enforced

1. **Datatype** – e.g., INT, VARCHAR, DATE
2. **NOT NULL** – cannot be left empty
3. **CHECK** – value must satisfy a condition
4. **DEFAULT** – assigns a default value
5. **UNIQUE** – no duplicates allowed

Syntax Example

```
CREATE TABLE Employee(
   emp_id INT,
   age INT CHECK(age >= 18),
   salary DECIMAL(10,2) NOT NULL,
   city VARCHAR(20) DEFAULT 'Ahmedabad'
);
```

Explanation

- Age must be **18 or above**
- Salary cannot be NULL

- City takes default value if not given

Domain integrity protects data accuracy and prevents wrong values like age = -5 or salary = NULL.

---

3. SQL Queries (Long Answer)

Given table: **BANK_MASTER(ename, balance, phno)**

1. Display all records with ename in descending order

```
SELECT * FROM BANK_MASTER ORDER BY ename DESC;
```

Purpose: Sorts employee names from **Z to A**.

---

2. Display employees having balance less than 5000

```
SELECT * FROM BANK_MASTER WHERE balance < 5000;
```

Purpose: Shows low-balance customers.

---

3. List employees who do NOT stay in Delhi

(Assuming city column exists)

```
SELECT * FROM BANK_MASTER WHERE city <> 'Delhi';
```

<> means NOT equal.

---

4. Display employee name starting with 'c'

```
SELECT * FROM BANK_MASTER WHERE ename LIKE 'c%';
```

`%` means any number of characters.

---

4. Identify SQL Functions. (Long Answer)

SQL functions are predefined operations that perform calculations on data and return a value.

Types of SQL Functions

1. **Aggregate Functions**
   Used with groups of rows.
   - `SUM()` – total value
   - `AVG()` – average value
   - `MAX()` – highest
   - `MIN()` – lowest
   - `COUNT()` – number of rows
2. **Scalar Functions**
   Work on single values.
   - `UPPER()` – convert to uppercase
   - `LOWER()` – lowercase
   - `LENGTH()` – find length
   - `ROUND()` – round value

Example

```
SELECT COUNT(*), AVG(salary) FROM employee;
```

Functions make SQL efficient by reducing multiple steps into one.

---

5. Prime Attribute & Functional Dependency (Long Answer)

Prime Attribute

A prime attribute is an attribute that is part of **any candidate key** of a relation.

- Candidate key = minimal set of attributes that uniquely identify a row.
- Prime attributes help identify tuples.

Example:
In Student(RollNo, Name, Class) → **RollNo** is prime.

Functional Dependency (FD)

FD is a relationship between attributes where one attribute **determines** another.

If A → B, it means **A uniquely decides B**.

Example

RollNo → StudentName
(If two students have same RollNo, name must be same → uniqueness)

FD helps in normalization.

---

6. Explain 3NF with example. (Long Answer)

Third Normal Form (3NF) is an important stage of database normalization that helps remove **transitive dependencies** and ensures that all attributes depend **directly on the primary key**, not on other non-key attributes.

Example:

*Table Not in 3NF

RollNo	Name	CourseID	CourseName
1	Amir	C1	DBMS
2	Asha	C2	Physics

*Table convert to 3NF

RollNo	Name	CourseID
1	Amir	C1
2	Asha	C2

CourseID	CourseName
C1	DBMS
C2	Physics

---

7. **Advantages and Disadvantages of Normalization (Long Answer)**

Advantages

1. **Removes data redundancy**
   Duplicate data is removed.
2. **Avoids anomalies**
   - Update anomaly
   - Insert anomaly
   - Delete anomaly
3. **Improves data integrity**
   Data remains correct, complete, and consistent.

Disadvantages

1. **More tables, more joins**
   Complex queries reduce performance.
2. **Slow retrieval**
   Joining multiple tables may slow down SELECT queries.

3. **Difficult design**
   Requires time, understanding, and proper planning.

---

8. Illustrate 2NF with example. (Long Answer)

2NF states that a table must be in 1NF and must not contain any partial dependency.
This means **every non-key attribute must depend on the whole primary key**, not on a part of it.

In Simple Words 2NF removes partial dependency from tables that have composite primary keys.

✕ Table NOT in 2NF

RollNo	Subject	SubjectName
1	DBMS	Database

Primary Key = (RollNo, Subject)
But **SubjectName** depends ONLY on **Subject**, not on the whole key → partial dependency.

So this table is **not in 2NF**.

☆ Convert to 2NF

✔ Table 1: Student_Subject

| RollNo | Subject |

✔ Table 2: Subject_Info

| Subject | SubjectName |

Now every column depends on the whole primary key, so it is in 2NF.

---

9. What is Two-Phase Locking Technique? (Long Answer)

Two-Phase Locking (2PL) is a **locking protocol** used in DBMS to maintain **serializability** between concurrent transactions.

It has two phases:

1. Growing Phase

- A transaction acquires locks (shared/exclusive)
- Cannot release any lock

2. Shrinking Phase

- After releasing a lock
- Cannot acquire any new lock

Why 2PL is needed?

- Prevents problems like lost updates, dirty reads
- Ensures that transactions appear to execute one after another

---

10. Problems of Concurrency Control with examples (Long Answer)

1. Lost Update Problem

Two transactions update the same data → one update gets overwritten.

Example:
T1 withdraws ₹100
T2 deposits ₹200
Final result may be incorrect.

---

2. Dirty Read

Transaction reads uncommitted data from another transaction.

Example:
T1 updates salary but not committed
T2 reads the updated salary
T1 rolls back → T2 read wrong data.

---

3. Unrepeatable Read

Same query returns different results because another transaction updated data.

---

4. Phantom Read

New records inserted by another transaction cause changed results.

Concurrency problems reduce reliability, so DBMS uses locks.

---

11. Note on Locking Mechanism with example (Long Answer)

Locking is a technique used to control concurrent access to data in a database.

Types

1. **Shared Lock (S-Lock)**
   - Used for reading
   - Many transactions can read at same time
2. **Exclusive Lock (X-Lock)**
   - Used for writing
   - No other transaction can read/write

Example

T1 wants to update Account 101 → X-Lock is applied
T2 wants to read Account 101 → must wait

Locks prevent conflicts and maintain consistency.

---

12. Transaction State Transition Diagram (Long Answer)

A database transaction goes through multiple states:

States

1. **Active** – Transaction is running
2. **Partially Committed** – All statements executed
3. **Committed** – Changes are permanently saved
4. **Failed** – Error occurred
5. **Aborted** – Rollback performed
6. **Terminated** – Transaction ends completely

The diagram shows movement from one state to another based on success or failure.

---

13. Identify ACID Properties (Long Answer)

ACID properties ensure reliable database transactions.

A – Atomicity

All steps of transaction are treated as a single unit.
Either **all happen** or **none**.

C – Consistency

After every transaction, the database must remain valid and follow all rules.

I – Isolation

Concurrent transactions should not affect each other.
Intermediate values must remain hidden.

D – Durability

Once a transaction is committed, changes are permanent even after system failure.

---

14. Convert table into 1NF (Long Answer)

1NF ensures that **every cell contains a single value only** and there are no repeating groups.

Example Table (Not in 1NF)

Name | Phone
Amit | 123, 456

Phone contains **multiple values** → violates 1NF.

Convert to 1NF

Break into separate rows:

Name | Phone
Amit | 123
Amit | 456

Now each cell has atomic (single) values only.
